A War Story About a Python, a Fork, a Cow, and a Bug

Or, why should you care about internals of your systems

ABHINAV UPADHYAY
DEC 17, 2023

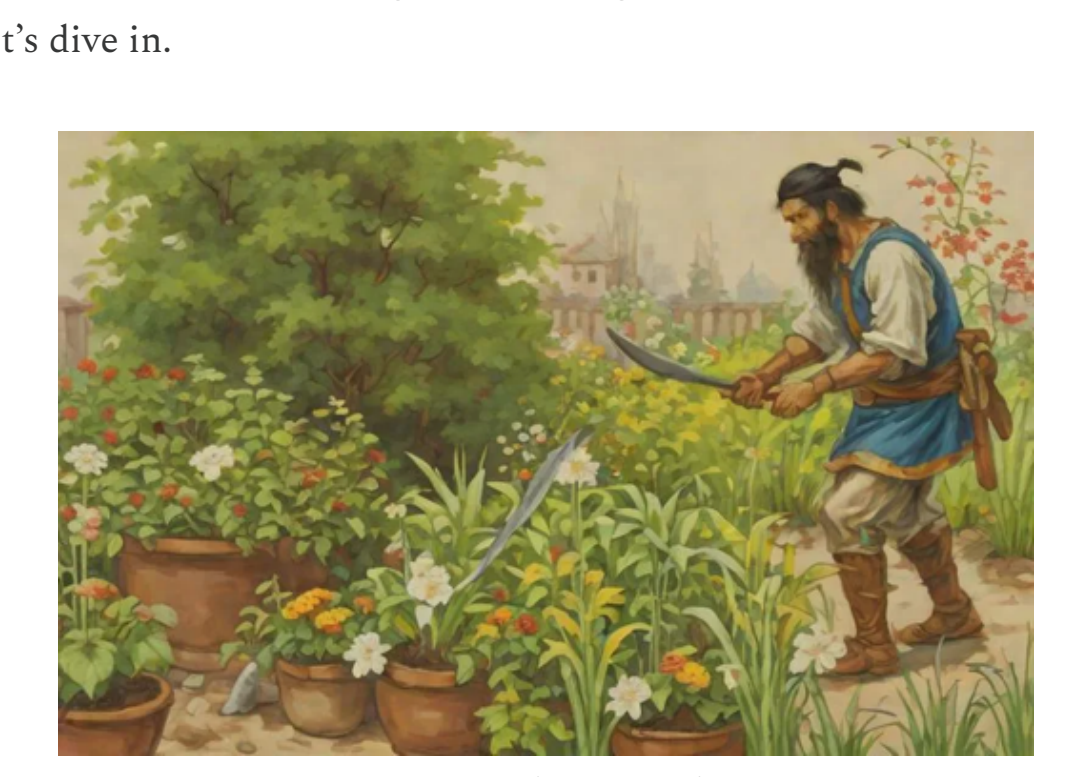
23 2 Share

> *"It is better to be a warrior in a garden, than a gardener in a war." — Miyamoto Musashi*

As software developers, you're likely comfortable writing code that solves complex business problems. But how often do you consider how things work under the hood of your programming language, database, or the operating system? I've been writing articles on CPython internals and I'm often asked why someone should invest time in understanding the internals of Python when the language's high-level features serve their immediate needs so well.

Although, there are several reasons that I could give which should persuade you – such as getting better at reading code, contributing to CPython, or growing overall as a developer by building a deep understanding of your stack. However, I didn't want to write a boring essay. So, to motivate you, I have an interesting story from the real-world. This should excite you for learning more about the internals of the systems that you use.

The story is about Instagram—how their engineers identified and fixed a serious performance issue in their production systems which wouldn't be possible if they didn't have a deep understanding of how things worked underneath their tech stack. Let's dive in.



A warrior working in a garden

Writing such in-depth articles on hard technical topics requires several days of research and writing. My goal is to build a community of 100 dedicated subscribers who value this work and want to ensure it continues. If you enjoy my content and would like to be part of this growing community, consider becoming a paid subscriber or buying me of a coffee. Every subscription takes us one step closer to our goal and keeps this work sustainable.

Type your email... Subscribe

The Case of Instagram's Spiking Memory Usage

It might be a surprise for a few people, but Instagram runs Python in production. They use a Django based service for serving their frontend. At their scale, they are very cautious about the performance of their services. As part of their monitoring, they identified a mysterious issue in their service. They noticed that their service's overall memory usage was unexpectedly going up with time.

None of their application code explained why this could happen, which meant this bug was present somewhere in the layer beneath their stack. This underlying layer consists of their web server, the Python runtime and the operating system (OS) itself.

While we could jump right to the end to discover the cause of this mysterious bug, let's walk through the investigation as if we were the ones uncovering it. We will start by analyzing Instagram's server setup, simply because their service is powered by a web server and if their application code is not responsible for consuming extra memory, then it could be a problem associated with the server.

Behind the Scenes: Instagram's Web Server Architecture

The job of a web server is to receive HTTP requests from a client (e.g. a browser) and return the requested resource (e.g. a web page, an API response, a CSS or JS file). However, even a modestly busy application can receive several requests per second, and if the server is handling them all within a single main process, the latency of the application would be far too high, because until it can finish processing one request other requests have to wait. To scale, the servers usually adopt one of the two strategies: pre-forking, or pre-threading.

In a pre-threading design, the server spawns and maintains a pool of several worker threads. Whenever the server receives a new request from the client, it assigns the request to one of the threads in the pool, or it puts the request in a queue which eventually gets picked by one of the threads.

The pre-forking architecture is similar, but threads are replaced by processes. The main server process spawns several child processes and the incoming requests are assigned to one of the worker processes. The following figure illustrates the pre-forking design, although in the figure you could replace processes with threads and you would get a pre-threading server.

An illustration of how a pre-forking web server operates. At any point of time there can be N incoming requests from the clients (depicted by web browsers). The web server assigns the requests to the worker processes, which do generate the response and return it back to the respective clients.

Generally, pre-threading is the preferred approach because threads are much light-weight than processes; they are faster to create and consume less resources. However, CPython does not have true thread level parallelism because of the global interpreter lock (GIL), which is why Instagram uses a pre-forking server (combined with asyncio, which allows them to scale well).

> *The GIL is a mutex that protects access to Python objects, preventing multiple threads from executing Python bytecodes at the same time. This means that even in a multi-threaded Python program, only one thread can execute Python code at any moment. While the GIL simplifies the management of CPython memory and native extension modules, it also means true parallel execution is not achievable with threads. Consequently, multi-threading in Python does not provide the performance benefits it does in languages without this limitation.*

Now, a pre-forking server needs to create a number of worker processes, for which it relies on the OS. It turns out the way the OS creates these processes and handles memory for those processes has details which might be pertinent to the bug. So let's dive into it.

Forking Processes and the CoW Mechanism

Instagram (and most of Meta) uses Linux as the OS for all their services. On Linux (and other Unix-like systems) the fork system call is commonly used to spawn new processes (although nowadays other mechanisms are available, such as posix_spawn or clone). The fork system call works by making a copy of the parent process, this means that the child process receives a copy of the parent's address space, inherits the parent's open file descriptors, and has the same process context (register values, program counter etc.).

Making a copy of all this data is expensive—especially copying the entire address space of the parent, as it can potentially be several megabytes. As an optimization, Linux employs the copy-on-write (CoW) technique.

CoW means that instead of copying the pages which are part of the parent's address space, the OS shares them between the parent and the child process in a read-only state. As long as the parent and the child process are not writing to those pages, the OS does not need to make a copy of those pages. However, if either the parent or a child process tries to mutate an object stored in a shared page, it would trigger CoW. This would mean that the OS needs to first make a copy of the page for each of the processes before performing the write.

Instagram takes advantage of the CoW optimization. In the main server process, they load and cache all the immutable state and data so that when the worker processes are forked, they each don't have to duplicate all of that. Which I have to say is a clever optimization.

Now, as part of their infrastructure monitoring, the Instagram engineers had the following graph showing the shared and private memory usage of the worker processes. Here private memory simply refers to the memory used by the worker processes which is not shared with any other process. The graph shows that over time the shared memory usage of the processes is going down while the private memory usage is going up. This clearly means that those shared pages are being modified, and triggering the CoW.

Image from the Instagram blog—*the effect of copy on writes is increasing private memory and a reduction of shared memory from the main process.*

We've come across our first clue—the shared pages between the main server process and the worker process are being modified. What could be the reason for this change? This modification could be happening in the application code. Checking the application code for patterns which might explain this behavior should be our next step. However, we can't do that here because we are not real Instagram engineers. Let's just skip past this step because Instagram engineers did this investigation and did not find any problem there.

This pushes us to look further down the stack. The next layer is the Python runtime because it is responsible for interpreting and executing all the code, and it's possible that the Python runtime does something behind the scenes which might cause modification of the read-only shared pages. Let's look at what happens inside the Python runtime.

The rest of the article is for paid subscribers. If you find my writing valuable and insightful, consider supporting it by becoming a paid subscriber. It will help me sustain and produce more content like this.

Python Runtime and How it Manages Memory

Could the Python runtime be behind the mysterious updates to read-only memory? To answer this we need to understand how it manages its memory.

CPython uses reference counting for managing memory of its runtime. Every object stores a count of the number of references which are pointing to it at any point of time. At the time of creation, an object starts with a reference count of 1 and every time it is assigned to another variable, or passed to a function, the reference count gets incremented. Similarly, when a variable goes out of scope, the reference count of the object it was pointing to goes down by 1. Finally, an object's memory is freed when its reference count reaches 0.

An illustration of how reference counts work behind the scenes in Python

Now, there are certain objects in Python which are constants and immutable, such as the values `True`, `False`, `None`. Even within the CPython implementation these are created as singleton objects, which means that every time we use `None` in our Python code, we are referring to the same object in memory.

It turns out that even though these objects are supposedly immutable, whenever we refer to them in our Python code, the CPython runtime would still increment their reference counts. For instance, when we write code like this:

```
if x is None:
    do_something()
```

the CPython runtime would modify the `None` object to increment its reference count.

Since these are immutable objects, they are part of the read-only pages shared between the worker processes. Even though the Instagram code has no Python code which is explicitly modifying these objects, the CPython runtime is updating them behind the scenes, leading to the modification of shared pages and triggering CoW.

This solves our mystery—the CPython runtime is the culprit, we don't have to drill down any further. However, this story would be incomplete if we did not see how it was fixed.

The Fix: Immortal Objects

Once the Instagram engineers figured out the problem, they proposed a fix to the CPython project. The fix was to mark immutable constant objects (such as True, False, None) in the CPython runtime as immortal. They proposed that the runtime treats these immortal objects especially, and does not attempt to update their reference counts. Thus these objects would become truly immutable. This proposal resulted in the PEP-683 which was accepted, implemented as part of the Python 3.12 release.

You might ask, how does this work internally? As I mentioned previously, each object stores a count of references to it in its in-memory representation. The trick to mark an object immortal is to set its reference count to a special constant value. The runtime simply checks if the reference count of an object is this special value, then it treats it as immortal and leaves it untouched.

> *If you are interested in more details behind this, I've an article which describes all the internals behind immortal objects, check it out.*

Conclusion: The Value of Digging Deeper

The story we just unpacked isn't just a tale of debugging; it's a confirmation of the importance of the ability to look beneath the surface of our high-level tools. The engineers at Instagram didn't just stumble upon a solution—they found it through a deliberate dive into the internals of CPython, coupled with a solid understanding of their entire stack.

It's clear from their experience that such issues, while not daily occurrences, are inevitable over a long enough timeline in production. The ability to dissect and understand these problems is what sets apart capable engineers from truly great ones.

Therefore, we should take a leaf out of Instagram's book and commit to understanding the systems we build upon a level deeper. By doing so, we ensure that when such bugs do present themselves, we're ready to tackle them head-on. Because, in the end, writing code isn't just about making things work—it's about knowing why they sometimes don't.

Resources

- Instagram blog post on the bug
- PEP-683 on Immortal objects

Enjoying these technical deep dives? Help sustain this work by becoming a paid subscriber. Join our mission to reach 100 subscribers—each one makes a difference. Support here with a subscription or by buying me a coffee. Thank you for being part of our community!

Type your email... Subscribe

Share

23 Likes · 2 Restacks

23 2 Share

← Previous Next →

Discussion about this post

Comments Restacks

Write a comment...

Top Latest Discussions

What Every Developer Should Know About GPU Computing
A primer on GPU architecture and computing
OCT 18, 2023 • ABHINAV UPADHYAY

How Many Lines of C it Takes to Execute a + b in Python?
Understand the mechanics of dynamic dispatch implementation in CPython
DEC 6, 2023 • ABHINAV UPADHYAY

Two Threads, One Core: How Simultaneous Multithreading Works Under the Hood
Ever wondered how your CPU handles two tasks at once? Discover the magic of Simultaneous Multithreading and see what's really going on inside.
JUL 24 • ABHINAV UPADHYAY

See all >

Ready for more?

Type your email... Subscribe

© 2024 Abhinav Upadhyay · Privacy · Terms · Collection notice

Start Writing Get the app

Substack is the home for great culture